

EVALUACIÓN GLOBALIZADORA B – TÉCNICAS DIGITALES III CONTROL DE ILUMINACIÓN

Eirea Martínez, Luis Alejandro
UTN FRA

Mehle, Ignacio
UTN FRA

Índice

1) Introducción	1
2) Objetivos	1
3) Diagrama en bloques.....	2
4) Alcance y fundamentación.....	2
5) Hardware.....	3
1.1. Microcontrolador.....	3
1.2. LUXOMETRO BH1750	3
1.3. EEPROM Y RTC	3
1.4. DISPLAY	3
1.5. ENCODER	4
1.6. SALIDA PWM.....	4
1.7. RASPBERRY PI.....	4
6).....	4
7) DIAGRAMA DE SOFTWARE.....	5
8) CONCLUSIONES.....	5

Abstract

Como Evaluación Globalizadora para la cátedra de Técnicas Digitales III, se presenta un sistema de control del nivel de iluminación en un ambiente, el cual debe poder setearse previamente, variando el brillo de tiras LED mediante PWM. El nivel de luz debe mantenerse estable en el nivel seteado frente a cambios en la iluminación. El sistema se puede ajustar de forma manual, y se plantea una interfaz remota en PC para seteo y monitoreo del nivel de luz, enlazada a la placa principal mediante una Raspberry Pi 4 corriendo un sistema Linux. Los seteos de iluminación en el tiempo se almacenan en un log de eventos accesible desde la PC. El controlador se basa en una placa de desarrollo Raspberry Pi Pico 2, utilizando un sistema operativo en tiempo real (FreeRTOS).

Palabras Clave

FreeRTOS – Control de Iluminación – Raspberry Pi Pico 2 – Interfaz remota

1) INTRODUCCIÓN

Como consigna de evaluación para la materia Técnicas Digitales III, se propone una controladora basada en una placa de

desarrollo Raspberry Pi Pico 2 [1], operativa de forma manual (primer cuatrimestre) y con acceso remoto mediante una interfaz gráfica (segundo cuatrimestre). El microcontrolador Pico 2 ejecuta el sistema operativo de tiempo real FreeRTOS, el cual comanda la ejecución del programa atendiendo a los eventos de control y operación de forma más eficiente y responsiva frente a una solución en baremetal [2].

2) OBJETIVOS

Se plantea un micro ambiente de prueba en forma de una pequeña caja, con una fuente de luz que consiste en tiras LED, las cuales se ajustan mediante PWM. La caja permite que la luz sea bloqueada parcialmente, para evaluar la respuesta del sistema a cambios en la iluminación.

Se busca poder setear el nivel de iluminación, y que el sistema reaccione ajustando el brillo de la tira LED para compensar la iluminación natural del ambiente y así lograr la iluminación deseada. Para llevar un registro de los seteos se recurre a un RTC DS3231, y a una memoria EEPROM AT24C32. En el menú se permite setear:

- NIVEL DE LUX
- ALARMA MAX
- ALARMA MIN
- CURVA DE AJUSTE

Las alarmas representan 2 niveles de iluminación frente a los cuales el sistema ya no puede compensar. Ambos pueden ser fijados por el usuario dentro de los rangos presentes en la calibración. Para esto se

implementan dos leds que indican las alarmas.

La curva de ajuste hace referencia a dos velocidades de respuesta de la controladora frente a las variaciones, rápida o de manera más gradual.

Para brindar un acceso remoto al proyecto, y como segunda parte de la evaluación globalizadora se usará una Raspberry Pi 4 que se conectará con la controladora ya sea físicamente (Ethernet) o no (WiFi) en donde correrá un Kernel de Linux que permitirá remotizar los seteos a y desde la memoria EEPROM así como también mostrar variables solicitadas en tiempo real. Esto se hará acompañado de una interfaz gráfica para escritorio, también en entorno Linux.

3) DIAGRAMA EN BLOQUES

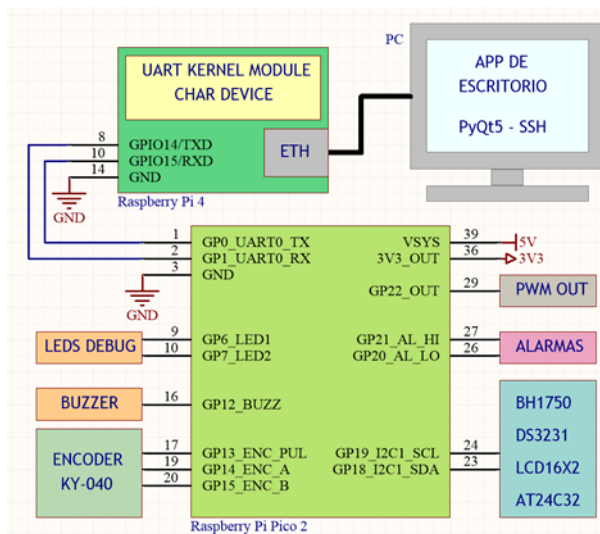


Figura 1 Diagrama en bloques

4) ALCANCE Y FUNDAMENTACIÓN

El alcance del proyecto fue definido como un control de iluminación a una tira led con un error menor a 5 lux.

El nivel de luz es medido con un luxómetro digital BH1750. Se emplea este módulo porque nos permite obtener una medición directa ya procesada del nivel de iluminación frente a la medición analógica con LDR, los cuales necesitarían un

contraste para trazar la curva de respuesta en función de la iluminación.

Luego, en los lineamientos de la evaluación se indicó la necesidad de guardar en forma de log de eventos los últimos seteos, y el momento en que fueron guardados, para lo cual se eligió un módulo de desarrollo basado en un RTC DS3231, el cual trae en el mismo PCB una memoria EEPROM AT24C32. Se decide emplear esta memoria porque con un tamaño de 32 Kbits nos permite almacenar más de 200 entradas en el log, lo cual es suficiente para el alcance del proyecto. Por otro lado, la idea de utilizar este módulo fue que se contaba con una librería funcional gracias a un proyecto anterior, la cual se adaptó al microcontrolador empleado.

Para visualización y control del sistema utilizamos de forma local un display LCD controlado por bus I2C, y un encoder con pulsador para la operativa manual. De esta manera tenemos una solución ágil y minimalista para el usuario.

Desde el punto de vista del firmware en la Pico 2, como lineamiento de la materia se utiliza el sistema operativo de tiempo real FreeRTOS. Se busca con esto lograr un sistema modular y responsivo frente a las tareas de control y la interacción con el usuario. Estas acciones, como la interacción con el hardware y la comunicación con la Pi 4, son modularizadas en Tareas del RTOS, cada una con una prioridad asignada.

Desde la Raspberry Pi 4, se brinda la capacidad de operar de manera remota la controladora, haciendo de interfaz física entre esta y la pc en donde se correrá una aplicación gráfica de escritorio. Para ello, se emplea una comunicación UART con la Pico 2. Desde dicha aplicación de se plantea:

- FECHAR EL RTC EN EL STARTUP
- GENERAR UN NUEVO SETTING DE ILUMINACIÓN Y AJUSTE

- MONITOREAR EL VALOR ACTUAL DE LUX
- LEER EL LOG DE SETTINGS EN EEPROM
- BORRAR EL LOG DE SETTINGS EN EEPROM

Las acciones de seteo de nivel de luz, lectura y operativa del log de settings se ejecutan mediante comandos enviados por SSH de la app de escritorio hacia la Pi 4, los cuales son enviados a la Pico 2 mediante un módulo de Kernel Linux, utilizando Char Device para el manejo de la UART [3][4].

La aplicación de escritorio se diseña usando la librería de Python PyQt5, con ayuda de la aplicación QtDesigner para el diseño gráfico y la librería paramiko para la comunicación vía protocolo SSH integrada [5][6].

Durante el desarrollo se observó que al ser la respuesta de la planta mucho más rápida (en este caso la respuesta de los leds al ciclo de actividad del PWM) en comparación a la velocidad de medición, se hace muy difícil el control usando el control PID.

Para resolver este problema se aplicó un filtro EMA (Exponential Moving Average) que ralentiza la respuesta de la planta, y se modificó la manera de medición de forma continua a mediciones sincronizadas cada 25 ms para asegurar un tiempo de muestra constante [7].

5) HARDWARE

1.1. Microcontrolador

Se emplea la placa de desarrollo Raspberry Pi Pico 2, la cual permite correr FreeRTOS como base del sistema. La placa cuenta con un microcontrolador RP2350 de dos núcleos Cortex M33 @ 150 MHz, unidad de punto flotante, 520kB de SRAM, 2MB de memoria flash, y utilizamos la extensión Raspberry Pi Pico de Visual Studio Code como IDE.

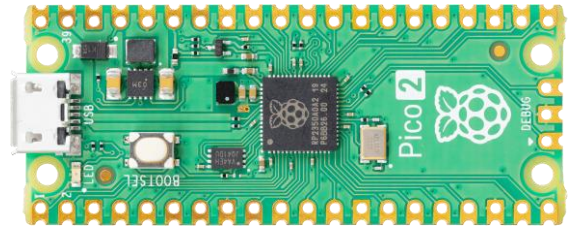


Figura 2 Raspberry Pi Pico 2

1.2. Luxómetro BH1750

El módulo BH1750FVI es un luxómetro digital con interfaz I2C. Se alimenta con 3,3V y entrega una lectura directa de la iluminación ambiente, con un rango de 1 a 65535 lx. Tiene una resolución de 1lx en modo High-Res (tiempo de medida de 120ms), y de 4lx en modo Low-Res (tiempo de medida de 16ms) [8].

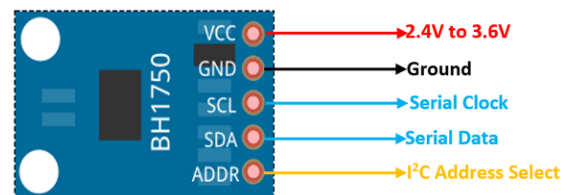


Figura 3 BH1750

1.3. EEPROM y RTC

Utilizamos un módulo que posee una EEPROM AT24C32 de 32kbits y un RTC DS3231 el cual es una actualización del más difundido DS1077, ya que puede alimentarse con 3,3V [9][10].

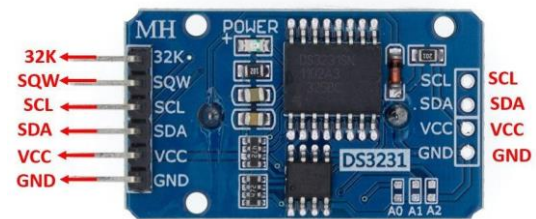


Figura 4 RTC / EEPROM

1.4. Display

Se emplea un display LCD de 16x2 que emplea el controlador HD44780, y junto a él utilizamos un expansor I2C a 8 bits serie/paralelo PCF8754, como interfaz I2C del LCD. Como el LCD se alimenta a 5V, y nuestro bus esta referenciado a los 3,3V que utiliza la Pico 2 y los demás módulos,

agregamos un convertor de niveles para poder utilizar el mismo bus.



Figura 5 Display LCD 16x2

1.5. Encoder

Para control por parte del usuario se empleó un encoder rotativo de 20 pasos KY-040, el cual posee un pulsador incorporado. Para facilitar la lectura de las transiciones, empleamos anti rebote físico con filtro RC y compuertas Schmitt Trigger para digitalizar los flancos y reducir la lógica de lectura.

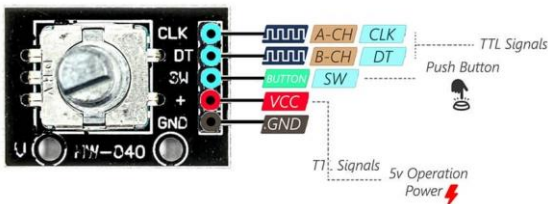


Figura 6 Encoder

Estas entradas se leen con interrupciones por hardware, que disparan tareas controladoras de FreeRTOS las cuales analizan la lógica del paso, si es horario o antihorario [11].

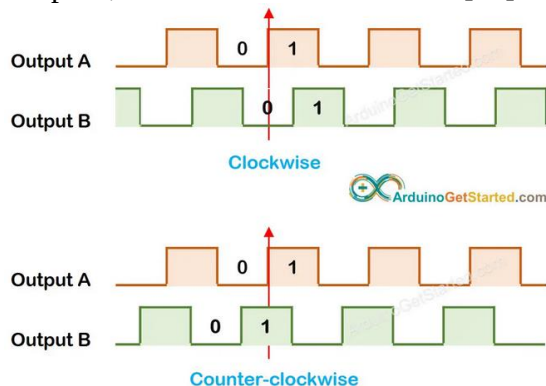


Figura 7 Transiciones del Encoder

1.6. Salida PWM

El control del brillo de la tira led se efectúa por medio de una etapa de potencia

MOSFET en corte saturación, usando un IRFZ44N de bajo $R_{DS(on)}$ y un driver tótem pole. Se agrega un buffer con filtro RC para medición del valor medio del PWM como testpoint.

1.7. RASPBERRY PI

El control remoto de la controladora se realiza gracias a una Raspberry Pi 4. Cuenta con una CPU Broadcom BCM2711, Quad Core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz, 4GB LPDDR4-3200 SDRAM, 2.4 GHz and 5.0 GHz IEEE 802.11ac wireless, Bluetooth 5.0, BLE Gigabit Ethernet, USB 2.0 y 3.0, 2 × micro-HDMI ports, entre otros periféricos.

La misma corre un sistema operativo Raspberry Pi OS Lite de arquitectura ARM64, y está basado en el Kernel Linux v6.12 [12]. La comunicación con la Pico 2 se realiza mediante la UART a niveles TTL. Por otro lado, la aplicación de escritorio en PC puede comunicarse mediante el protocolo SSH bien mediante Ethernet o habiendo vinculado previamente la Pi 4 a una red WiFi.



Figura 8 Raspberry Pi 4

6) LISTADO DE COMANDOS (CLI)

>> get lux

Trae el valor actual de lux medido por el luxómetro y los settings actuales. Ej:
[NOW] LUX 1055 MAX 1200 MIN 800 CURVA 0

>> set lux -v int -h int -l int -a int

Genera un nuevo setting de control. Los argumentos son:

- v = valor de lux
- h = alarma high
- l = alarma low
- a = curva de ajuste (0: rápida, 1: lenta)

Ej: *set lux -v 1000 -h 1100 -l 900 -a 0*

>> get log

Imprime el log de settings actual en EEPROM. El log de settings se guarda en el archivo "log.txt".

>> set eclear

Borra los settings en EEPROM

>> set rtc dd/mm/aa wd hh:mm:ss

Fecha el RTC. Ej:

set rtc 10/12/25 3 12:30:00

7) DIAGRAMA DE SOFTWARE

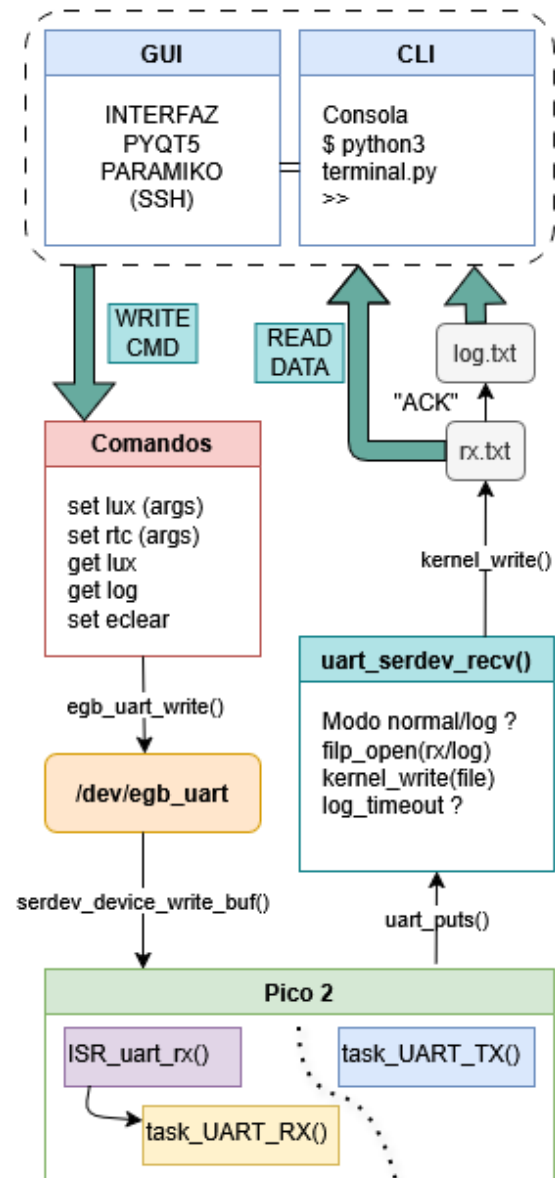


Figura 9 Diagrama de Software

8) CONCLUSIONES

Se pudo lograr un sistema que ajusta el brillo de la luminaria (leds) adecuándose a la iluminación requerida por el usuario dentro de los niveles de ajuste que brinda el microcontrolador y la luminaria dada.

Se pudo lograr que el sistema guarde los seteos en la memoria EEPROM e implementar la funcionalidad de leer los settings guardados en el tiempo y borrarlos. De forma empírica se pudo llegar a 2 ajustes de respuesta de la planta, uno rápido y uno lento, que dependen del filtrado aplicado al PWM, utilizando un control PI en cada caso. Se observó que para la respuesta rápida la adición de la acción derivativa no arrojaba mejoras sustanciales.

Tuvimos muchos problemas para poder utilizar el UART en la Pi 4, dado que el soporte para Serial Device (serdev) está desactualizado en los kernels nuevos, por conflicto con protocolos de bluetooth y consola serie de Linux, lo que ocasiono contratiempos.

Se pudo lograr un control de la EGA de forma remota desde la aplicación de terminal CLI en Python, y mediante una aplicación grafica utilizando PyQT5. Pueden mejorarse bugs menores en el fechado, lectura del encoder, parseo de datos y/o generación de archivos en la CLI e implementar un script de instalación automática del módulo de kernel, archivos y apps, así como una mejora en el login desde la GUI para no depender de la IP física asignada a la Pi. También se podrían exportar las apps como standalones para no depender de una instalación de Python en la Pi y en la PC remota.

Referencias

- [1] Raspberry Pi Pico 2 SDK:
https://www.raspberrypi.com/documentation/microcontrollers/c_sdk.html
- [2] FreeRTOS
<https://www.freertos.org/>
- [3] Char Device:
https://linux-kernel-labs.github.io/refs/heads/master/labs/device_drivers.html
- [4] Char Device:
<https://tldp.org/LDP/lkmpg/2.4/html/c577.htm>
- [5] Primeros Pasos con Qt Designer y PyQt5:
<https://www.pythonguis.com/tutorials/first-steps-qt-creator/>
- [6] Paramiko:
<https://www.paramiko.org>
- [7] Apuntes de Cátedra de Electrónica Industrial
<https://leifrautn.blogspot.com/search/label/Electr%C3%B3nica%20industrial%201>
- [8] BH1750:
<https://www.alldatasheet.es/datasheet-pdf/view/338083/ROHM/BH1750FVI.html>
- [9] DS3231:
<https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>
- [10] AT24C32:
<https://ww1.microchip.com/downloads/en/DeviceDoc/doc0336.pdf>
- [11] KY 040:
<https://eloctavobit.com/modulos-sensores/codificador-rotatorio-ky-040-rotary-encoder>
- [12] Linux Embebido:
<https://elixir.bootlin.com/linux/v6.12/source>

ANEXO 1 – GUÍA DE TAREAS

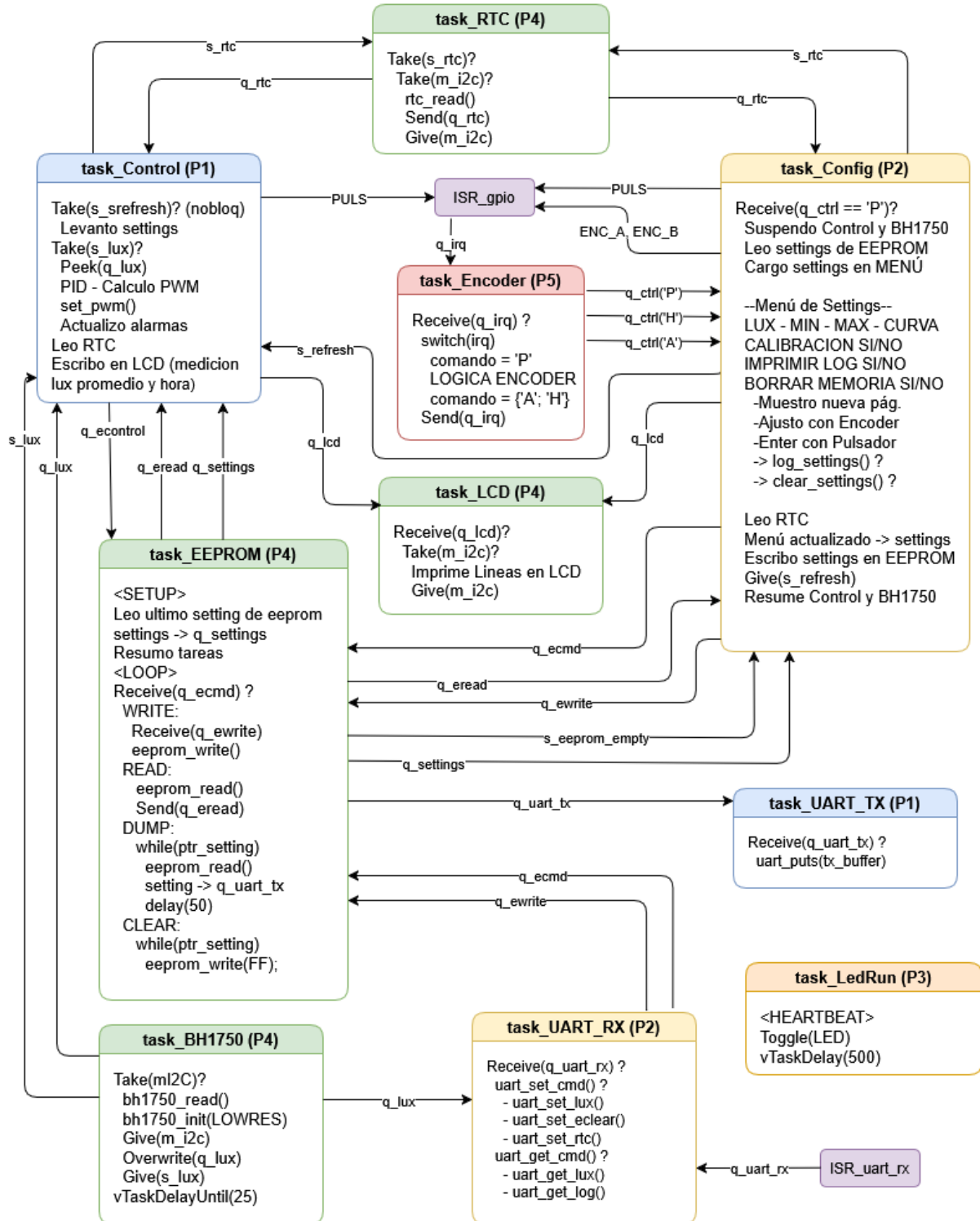


Figura 10 Diagrama de Tareas - FreeRTOS